ChallengeHub Integrated Software Design Portfolio

Course: CPE106L-4 Software Design Laboratory (FOPI01) | Team: Team01
Team Roster: Rachel Joy Baldo (PM) | Mark Angelo Aycardo (Architect) |
Jian Bernard Quinton (Git Master) | Issa Dumlao (QA Lead)

Section 1: Problem, Scope, and Requirements

1.1 System Context & Problem Statement:
Virtual wellness and productivity tracking applications often suffer from unvalidated state mutations, progress counter drift, and tight coupling between user interfaces and underlying persistence engines. ChallengeHub resolves these architectural issues by providing an event-driven system enforcing strict in-memory precondition validations, zero state mutation on failure, and dynamic progress derivation.

1.2 Stakeholder & Boundary Analysis:

- Organizer: Creates and manages wellness/productivity challenges (FB-01), defining active calendar dates, allowed activity types, and target completion goals.
- Participant: Enrolls in active challenges (FB-02) and records activity logs (FB-03). Demands transparent, real-time feedback on progress metrics (FB-05).
- System Scope Boundary: Enforces strict layered decoupling (CLI Presentation -> Application Coordinator -> Domain Core -> Infrastructure Persistence) with unidirectional downward dependency flow.

1.3 Functional & Non-Functional Requirements:

Req ID	Type	Requirement Description & Operational Contract	Traced Workflow
FB-01	Functional	Organizers create challenges with goal units, target amounts, and active calendar date bounds.	UC-01 Manage Challenge
FB-02	Functional	Participants enroll in active challenges prior to submitting activity log entries.	UC-02 Enroll Challenge
FB-03	Functional	System records activity entries after passing 6 strictly ordered in-memory precondition checks.	UC-05 Record Activity
FB-04	Functional	Failure Invariant: Rejected submissions trigger zero state mutation in memory lists or physical disk files.	UC-05 Record Activity
FB-05	Functional	System dynamically derives progress totals using Strategy pattern policies without stored progress columns.	UC-04 View Progress
FB-06	Functional	System generates structured participant progress summaries and administrative review views.	UC-06 Review Summary
FB-07	Functional	Atomic write-through persistence guarantees disk durability upon valid activity entry.	UC-05 Persistence
NFB-01	Non-Functional	Layered decoupling: CLI handles presentation while ChallengeManager manages orchestration.	Architectural Layering
NFB-02	Non-Functional	Informative error handling: Rejections display clear error messages without application crashes.	Error Handling
NFB-03	Non-Functional	Execution idempotency: Verification test suite executes cleanly in isolated temporary directories.	Verification Isolation

Section 2: Analysis and Behavioral Design

2.1 Use-Case Specification: UC-05 Record Activity

- Primary Actor: Participant
- Preconditions: Participant exists, Challenge exists,Active enrollment pairing exists.
- Main Success Scenario: Participant submits activity parameters → CLI invokes ChallengeManager.record_activity() → In-memory preconditions pass → Unique record_id generated → ActivityRecord instantiated → Appended to memory list → Atomic write-through via storage.save_activities() → Dynamic progress computed → CLI renders success confirmation.
- Failure Scenario (FB-04): Precondition check fails (e.g. negative amount or date out of bounds) → System raises ValueError → Execution halts immediately → Memory list and CSV file remain 100% unmutated → CLI displays user-friendly rejection notice.

2.2 Precondition Guard Evaluation Order:

To guarantee zero state mutation on failure, ChallengeManager.validate_preconditions() evaluates six distinct memory guards in strict sequential order prior to object instantiation or disk dispatch:

1. Participant ID exists in registered participants dictionary.
2. Challenge ID exists in registered challenges dictionary.
3. Active enrollment pairing (participant_id, challenge_id) exists in enrollments list.
4. activity_type matches challenge.activity_type (case-insensitive).
5. amount > 0 (strictly positive float).
6. activity_date falls within [challenge.start_date, challenge.end_date] inclusively.

Section 3: User and System Design

3.1 CLI Context of Use & Interaction States:

The Command Line Interface (challengehub/cli.py) provides an interactive menu loop. Users transition across three distinct states: (1) Prompt Selection, (2) Parameter Entry Form, and (3) Response / Error State. Rejections trigger clear, formatted error messages without breaking the interactive CLI loop.

3.2 Layered System Architecture:

The software architecture enforces a strict 3-layer structure with unidirectional downward dependencies:

- Presentation Layer (challengehub/cli.py): Handles command parsing and output formatting.
- Application Coordination Layer (challengehub/manager.py): ChallengeManager orchestrates use-case workflows and precondition validation.
- Domain Core Layer (challengehub/models.py & strategies.py): Pure domain dataclasses and strategy policies.
- Infrastructure Persistence Layer (challengehub/storage.py): Procedural JSON and CSV I/O routines.

Section 4: Detailed Design

4.1 Operational Contract: UC-05 Record Activity

- Operation: record_activity(participant_id, challenge_id, activity_type, amount, activity_date, note: str = "")

- Inputs: participant_id (str), challenge_id (str), activity_type (str), amount (float), activity_date (date/str), note (str)
- Preconditions: All 6 memory validation guards pass.
- Postconditions: New ActivityRecord instantiated; appended to self.activities list; save_activities() serializes updated list to activities.csv; returns created record.

4.2 Structural Class Specifications & Type Signatures:

Class / Entity	Stereotype / Role	Attributes & Type Signatures	Key Methods & Responsibilities
Participant	@dataclass Entity	participant_id: str name: str email: str	Pure domain entity representing a registered user.
Challenge	@dataclass Entity	challenge_id: str name: str activity_type: str goal_unit: str goal_target: float start_date: date end_date: date	is_active(current_date) → bool: Evaluates calendar date bounds.
Enrollment	@dataclass Entity	participant_id: str challenge_id: str joined_on: date	Models active participation pairing.
ActivityRecord	@dataclass Entity	record_id: str participant_id: str challenge_id: str activity_type: str amount: float activity_date: date note: str = ''	to_dict()→ dict: Formats record for CSV persistence.
ChallengeManager	Application Coordinator	data_dir: Path participants: dict challenges: dict enrollments: list activities: list _strategies: dict	record_activity(...)→ ActivityRecord progress(participant_id: str, challenge_id: str) -> dict _validate_preconditions(...) _get_strategy(...)
ProgressStrategy	«interface, ABC»	Abstract Base Strategy	calculate(records: list, challenge: Challenge) → float {abstract}
AmountProgressStrategy	Concrete Strategy	Inherits ProgressStrategy	calculate(...): Returns round(sum(r.amount), 2) for distance metrics.
CountProgressStrategy	Concrete Strategy	Inherits ProgressStrategy	calculate(...): Returns float(len(records)) for discrete session metrics.
storage.py	«module» Persistence	Procedural functions	load_participants(), load_challenges(),

			load_enrollments(), load_activities(), save_activities()

4.3 Strategy Pattern ADR & Rejected Alternatives:

- Problem Addressed: Fitness challenges define fundamentally different metric calculation models-distance totals (km/miles) vs. discrete session counts. Hardcoding conditional checks inside ChallengeManager violates the Open/Closed Principle.
- Selected Strategy Pattern: Encapsulates metric algorithms inside the ProgressStrategy hierarchy, satisfying the Open/Closed Principle. New metric policies can be added without mutating ChallengeManager.
- Rejected / Deferred Alternatives: Factory Pattern was rejected as decorative indirection because all activity records share a single homogeneous ActivityRecord schema. Observer Pattern was deferred to Module IS as unnecessary overhead for single-user CLI.

Section 5: Data, Output, and Verification

5.1 Canonical Baseline Benchmark & Data Reconciliation:

The system reads flat-file persistence datasets stored under data/: participants.json, challenges.json, enrollments.json, and activities.csv.

Authoritative Baseline Benchmark (Team01):

- Challenge C001 (Walk 20K): activity_type = 'walk', goal_unit = 'km', goal_target = 20.0 km.
- Participant P001: Enrolled in C001. Baseline activity entries in activities.csv are AR-0001 (3.5 km) and AR-0002 (4.0 km).
- Canonical Initial Progress: 3.5 + 4.0 = 7.5 km = 37.5% completion (7.5 / 20.0 km).

Reconciliation Note: Early PO4 draft narrative prose provisionally cited a 10.0 km target. In this final reconciled baseline, all narrative prose is aligned with canonical challenges.json (20.0 km target) without dataset mutation.

5.2 Verification Matrix & Runtime Evidence:

Verification Case	Target Requirement	Stimulus & Execution Details	Runtime Observed Evidence	Status
Normal Path Execution	FB-01, FB-02, UC-05	record_activity('P001', 'C001', 'walk', 2.0, date(2026, 8, 10))	ActivityRecord AR-0004 created; record count 3 → 4; progress 7.5 → 9.5 km (47.5% completion); CSV updated.	PASSED
Boundary Valid Path	FB-03, UC-05	record_activity('P001', 'C001', 'walk', 0.01, date(2026, 8, 1))	ActivityRecord AR-0005 created on start date boundary; record count 4 → 5; progress 9.5 → 9.51 km.	PASSED
Failure Invariant Check	FB-04, NFB-02	record_activity('P001', 'C001', 'walk', 0.0, date(2026, 8, 10))	Raises ValueError: Amount must be positive; record count strictly 5 → 5; progress unchanged at 9.51 km.	PASSED
Synthetic Probe Isolation	NFB-03	Executing test suite in isolated temporary directories (tempfile)	Zero side-effects on primary production data files; clean setup/teardown execution.	PASSED

Persistence Reload Coherence	FB-07	Instantiating fresh ChallengeManager reloading from disk	Restores exactly 5 activity records and derived progress of 9.51 km matching memory state.	PASSED

Section 6: Traceability and Design Evolution

6.1 Requirements-to-Code-to-Test Cross-Model Traceability Matrix:

Req ID	Requirement Statement	Use Case / Contract	UML Location	Python Implementation	Pytest Verification
FB-01	Create & manage challenges	UC-01	Pages 1, 2, 5	Challenge, storage.load_challenges	test_baseline_data_loading
FB-02	Participant enrollment	UC-02	Pages 1, 2, 5	Enrollment, storage.load_enrollments	test_baseline_data_loading
FB-03	Record activity log entries	UC-05	Pages 2, 3, 5	ChallengeManager.record_activity	test_normal_path_activity_recording
FB-04	Failure Invariant (Zero Mutation)	UC-05	Pages 2, 3, 5	_validate_preconditions	test_zero_state_mutation_on_failure
FB-05	Dynamic progress calculation	UC-04	Pages 2, 4, 5	ProgressStrategy hierarchy	test_normal_path_activity_recording
FB-06	Structured summaries & reviews	UC-06	Pages 1, 5	ChallengeManager.progress, CLI	test_normal_path_activity_recording
FB-07	Atomic write-through persistence	UC-05	Pages 2, 3, 5	storage.save_activities	test_persistence_reload_coherence
NFB-01	Layered decoupling	Architecture	Pages 1, 5	cli -> manager -> models/storage	test_layered_decoupling
NFB-02	Informative error handling	CLI Loop	Pages 3, 5	CLI exception catching	test_zero_state_mutation_on_failure
NFB-03	Verification isolation	Testing	Page 5	pytest tempfile fixtures	test_suite_isolation

6.2 System Design Evolution Summary:

Throughout the development lifecycle, Team01 executed three major model and architectural reconciliations:

1. Reconciliation of Baseline Goal Metrics: Audited early PO4 narrative drafts and harmonized all goal target references to match canonical challenges.json (20.0 km target for C001).
2. Refactoring to Strategy Pattern: Replaced hardcoded type conditionals with the ProgressStrategy hierarchy to adhere to the Open/Closed Principle.
3. Two-Phase Precondition Validation: Introduced _validate_preconditions() to evaluate all 6 memory guards prior to state mutation, guaranteeing FB-04 zero mutation.

Section 7: Limitations and Responsible AI Use

7.1 System Limitations & Module IS Revision Target:

- Current Prototype Limitations:
  (1) Single-participant visualization scope in CLI charts; (2) Procedural storage routines tightly coupled to flat CSV/JSON files lacking true ACID transaction locks.
- Module IS Revision Targets:
  (1) Migrate persistence to an embedded SQLite relational database; (2) Introduce a formal ActivityRepository interface to decouple storage from application logic; (3) Expand CLI views to support multi-participant comparative analytics.

7.2 Responsible AI-Use Disclosure:

What We Asked AI to Do	Consulted AI for structuring multi-layer pandas DataFrame validation checks, progression visualizer templates, fresh-extraction pytest scripts
Useful Advice / Output Used	Recommended separating data validation into non-mutating audit passes to prevent silent row dropping; suggested an automated clean-room extraction test using Python tempfile and zipfile modules
How We Verified It	Executed all unit tests and assertion cells inside isolated temporary directories; verified cumulative arithmetic independently (3.5 + 4.0 = 7.5 km); confirmed via disk file inspection that zero state mutations occurred on rejected inputs
What We Corrected / Rejected	Corrected date comparison logic to discrete calendar dates (datetime.date) to prevent timestamp clipping; rejected external save adapters because ChallengeManager persists atomically on write; rejected decorative Factory and Observer abstraction
Final Human Team Decision	Retained PID='P001' and CID='C001' across all notebook acts to maintain end-to-end traceability; preserved the strict 3-layer architecture with dynamic Strategy-derived progress; froze the verified write-through baseline.

Section 8: Professional and Ethical Responsibility

8.1 Exposed Ethical Responsibility: Data Validity and Zero-Mutation Integrity:
In digital wellness and competitive health tracking platforms, data integrity directly impacts user trust and equity. Allowing unvalidated, out-of-bounds, or corrupted activity entries undermines fair competition and distorts fitness progress tracking.

8.2 Architectural Safeguards and Impact Analysis:
Team01 addressed this ethical responsibility through three core architectural safeguards:

- In-Memory Validation Guards: ChallengeManager._validate_preconditions() validates inputs before state mutation
- Guaranteed Zero Mutation (FB-04): Rejected requests trigger an immediate ValueError, leaving memory lists and CSV files 100% unmutated
- Dynamic Progress Derivation: Progress is computed on demand from immutable accepted logs, eliminating state drift and leaderboard tampering